

Ejercicio Final JavaScript – Sistema de Gestión de Tickets

Objetivo

Desarrollar una aplicación web que permita gestionar **tickets de soporte técnico**, implementando:

- Autenticación mediante un servicio independiente
- Manejo de roles (Administrador y Técnico)
- Persistencia de sesión
- CRUD de tickets
- Consumo de APIs simuladas con **json-server**

Contexto del problema

Una empresa de soporte técnico necesita un sistema interno para gestionar incidencias reportadas por los usuarios.

El sistema debe permitir que los técnicos registren tickets y que los administradores gestionen la asignación de los mismos.

Roles del sistema

Administrador (admin)

Puede:

- Ver todos los tickets
- Crear tickets
- Editar tickets
- Eliminar tickets
- Asignar técnico responsable a cualquier ticket
- Ver todos los usuarios técnicos
- Asignar estado del ticket (En proceso, Asignado, Solucionado) se podrá seleccionar el estado asignado solo cuando se asignó un técnico.



Técnico (tecnico)

Puede:

- Crear tickets
- Ver los tickets creados por él
- Editar sus propios tickets
- Asignar estado del ticket (En proceso, asignado, Solucionado)

Restricciones:

- No puede asignar técnicos
- Al crear un ticket se asigna automáticamente como responsable

Cliente (cliente)

Puede:

- Crear tickets
- Ver los tickets creados por él
- Editar sus propios tickets siempre y cuando no tengan un técnico asignado o si el estado del caso es igual a cerrado

Restricciones:

- No puede asignar técnicos
- Al crear un ticket se asigna automáticamente como responsable

1. Autenticación

La aplicación debe incluir:

- Formulario de login
- Validación de credenciales contra un **json-server de autenticación**
- Manejo de errores

Importante

Debe existir **dos servicios json-server separados**:

1. **Servicio de autenticación**
 - Usuarios
 - Roles
 - Login simulado
2. **Servicio de datos**
 - Tickets
 - Técnicos
 - Información de negocio

2. Funcionalidad de Usuarios

- Debe existir un sistema de registro de usuarios del sistema, el cual permitirá que los clientes se registren y queden automáticamente con el rol Cliente
- El sistema debe iniciar con usuarios de tipo administrador y técnicos cargados en el json
- Debe haber como mínimo:
 - 1 administrador
 - 2 técnicos

3. Gestión de Tickets

Cada ticket debe contener:

- Nombre del ticket
- Tipo de caso (ej: incidente, requerimiento, soporte)
- Descripción
- Técnico asignado
- Cliente que solicita que realiza el reporte

4. CRUD de tickets

Se debe implementar:

- GET → Obtener tickets
- POST → Crear tickets
- PUT/PATCH → Editar tickets
- DELETE → Eliminar tickets (solo admin)

5. Reglas de negocio

- Si un **técnico crea un ticket**:
 - Se asigna automáticamente como responsable
- Si un **administrador crea o edita un ticket**:
 - Puede asignar cualquier técnico existente en el sistema
- Si un **cliente crea un ticket**:
 - No podrá seleccionar un técnico, solo podrá registrar el ticket y esperar que un administrador asigne un técnico
 - El cliente podrá ver todos sus tickets y editarlos siempre y cuando aún no tenga un técnico asignado

6. Protección de rutas

Debe incluir:

- Restricción de acceso según autenticación
- Restricción por rol:
 - Técnico o un cliente no puede acceder a funcionalidades administrativas
- Redirecciones automáticas
- Logout funcional
- Si se intenta ingresar a una ruta del sistema dedicada a los usuarios con rol administrador sin serlo, se deberá mostrar un mensaje de acceso denegado, lo mismo ocurrirá si un cliente intenta ingresar a la url dedicada a los técnicos

7. SPA (Single Page Application)

La aplicación debe:

- No recargar la página
- Tener navegación dinámica
- Renderizar el contenido mediante JavaScript

8. Interfaz de usuario

Puede usarse:

- CSS
- Tailwind
- Bootstrap
- Cualquier librería de estilos

Se evaluará:

- Organización visual
- Experiencia de usuario
- Responsividad
- Claridad en la interfaz

9. Persistencia de sesión

Debe implementarse usando:

- localStorage / sessionStorage / cookies

Requisitos:

- Mantener sesión al refrescar
- Logout funcional mediante botón de cerrar sesión
- Logout funcional mediante tiempo, es decir pasado 5 minutos de inactividad se debe cerrar la sesión automáticamente
- Limpieza de sesión

10. Documentación (Inglés obligatorio)

Debe incluir un archivo **README.md** con:

- Project name
- Description
- Technologies used
- Installation
- How to run both json-server services
- Project structure
- Role behavior explanation
- Technical decisions

Requisitos técnicos

Debe incluir:

- Código modular (Capaz de ser utilizado en otros proyectos con gran facilidad)
- Manejo de eventos
- Consumo de APIs mediante axios
- Manejo de errores
- Código limpio y organizado
- Uso de middleware simulados para la funcionalidad de protección de rutas y autenticación

Uso de json-server

Se deben levantar **dos instancias independientes**:

Ejemplo:

Autenticación

```
json-server --watch auth-db.json --port 3001
```

Datos de la aplicación

```
json-server --watch data-db.json --port 3002
```

Entregables

Se deberá realizar en grupos de máximo 4 Coders, los cuales deben entregar:

- Proyecto completo
- `auth-db.json`
- `data-db.json`
- README.md con el detalle del proyecto y sus integrantes
- (Opcional) evidencias visuales
- Comprimido en formato zip cargado en Moodle, sin la carpeta nodemodules
- Indicar quién será el líder exponente de la solución
- Cada integrante debe cargar su entrega en Moodle.



Cronograma

- **Martes:** Inicio del desarrollo
- **Miércoles:** Finalización del desarrollo
- **Jueves 4 de junio:** Sustentación técnica

Restricciones

Prohibido:

- Uso de IA generativa
- Copiar proyectos completos
- Uso de plantillas CRUD listas

Permitido:

- Documentación oficial
- MDN
- StackOverflow
- Recursos educativos

Mensaje final

El enfoque está en la lógica, arquitectura y manejo real de escenarios.

Diseña pensando como si fuera un sistema productivo real.